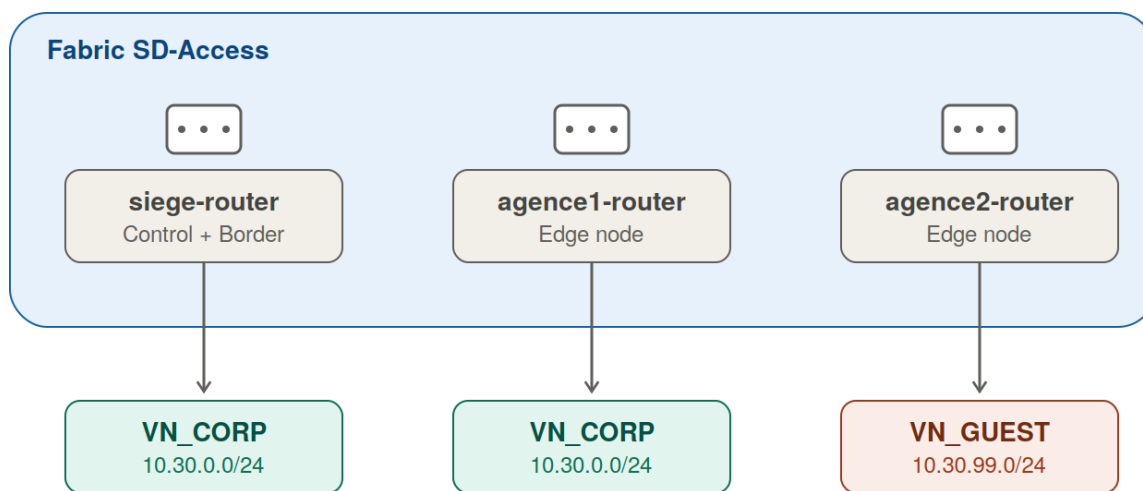


TP16 — Déployer une fabric SD-Access simulée

Formation NetOps : Automatiser la gestion de son Infrastructure Réseau — Ambient IT

Description



Fleche = onboarding automatisé de l'hôte sur GigabitEthernet0/1

Figure 1: Fabric SD-Access à déployer

SD-Access est l'architecture réseau logicielle de Cisco qui organise un ensemble d'équipements en fabric : des nœuds jouant des rôles précis (contrôle, bordure, périphérie), des réseaux virtuels (VN) isolant les usages (corporate, invité), et un onboarding automatisé qui rattache chaque hôte au bon réseau virtuel sans configuration VLAN manuelle par site. Ce TP étend le mock DNA Center du TP15 pour simuler ces concepts, déploie une fabric à 3 sites, onboarde des hôtes, et détecte automatiquement un nœud en panne (troubleshooting).

Prérequis

- TP1 à TP15 validés

Étapes

Étape 1 — Créer sa branche

```
cd netops-project-<nom-groupe>
git checkout dev
git pull origin dev
git checkout -b <prenom>-<nom>-jour4-tp16
```

Étape 2 — Étendre le mock DNA Center avec les endpoints SD-Access

```
cat > scripts/mock_dnac_api.py << 'EOF'
import time
import uuid

from flask import Flask, request, jsonify

app = Flask(__name__)

VALID_USER = "admin"
VALID_PASS = "dnacpass123"
VALID_TOKEN = "dnac-fake-token-xyz"

DEVICES = [
    {"id": "dev-siege", "hostname": "siege-router", "managementIpAddress": "10.10.10.1", "reachabilityStatus": "Reachable"},
    {"id": "dev-agence1", "hostname": "agence1-router", "managementIpAddress": "10.10.10.2", "reachabilityStatus": "Reachable"},
    {"id": "dev-agence2", "hostname": "agence2-router", "managementIpAddress": "10.10.10.3", "reachabilityStatus": "Reachable"},
]

FABRIC_ROLES = {
    "dev-siege": ["CONTROL_PLANE_NODE", "BORDER_NODE"],
    "dev-agence1": ["EDGE_NODE"],
    "dev-agence2": ["EDGE_NODE"],
}

TASKS = {}
FABRIC_SITES = {}
ONBOARDED_HOSTS = []

VALID_POOLS = {"VN_CORP": "10.30.0.0/24", "VN_GUEST": "10.30.99.0/24"}

def check_auth_token():
    return request.headers.get("X-Auth-Token") == VALID_TOKEN

def find_device(device_id):
    return next((d for d in DEVICES if d["id"] == device_id), None)
```

```

@app.route("/dna/system/api/v1/auth/token", methods=["POST"])
def auth_token():
    auth = request.authorization
    if not auth or auth.username != VALID_USER or auth.password != VALID_PASS:
        return jsonify({"error": "Unauthorized"}), 401
    return jsonify({"Token": VALID_TOKEN})

@app.route("/dna/intent/api/v1/network-device", methods=["GET"])
def network_device():
    if not check_auth_token():
        return jsonify({"error": "Invalid or missing X-Auth-Token"}), 401
    return jsonify({"response": DEVICES})

@app.route("/dna/intent/api/v1/template-programmer/template/deploy", methods=["POST"])
def deploy_template():
    if not check_auth_token():
        return jsonify({"error": "Invalid or missing X-Auth-Token"}), 401
    body = request.get_json(force=True)
    if not body or "templateId" not in body or "targetInfo" not in body:
        return jsonify({"error": "templateId et targetInfo sont requis"}), 400
    task_id = str(uuid.uuid4())[:8]
    TASKS[task_id] = {"polls": 0}
    return jsonify({"deploymentId": f"dep-{task_id}", "taskId": task_id})

@app.route("/dna/intent/api/v1/task/<task_id>", methods=["GET"])
def get_task(task_id):
    if not check_auth_token():
        return jsonify({"error": "Invalid or missing X-Auth-Token"}), 401
    if task_id not in TASKS:
        return jsonify({"response": {"progress": "Unknown task", "isError": True}}), 404
    TASKS[task_id]["polls"] += 1
    if TASKS[task_id]["polls"] < 3:
        return jsonify({"response": {"progress": "IN_PROGRESS", "isError": False, "endTime": None}})
    return jsonify({"response": {"progress": "TEMPLATE_DEPLOYED", "isError": False, "endTime": int(time.time() * 1000)}})

@app.route("/dna/intent/api/v1/business/sda/fabric-site", methods=["POST"])
def create_fabric_site():
    if not check_auth_token():
        return jsonify({"error": "Invalid or missing X-Auth-Token"}), 401
    body = request.get_json(force=True)
    site_name = body.get("siteNameHierarchy")
    device_id = body.get("deviceId")

```

```

if not site_name or not device_id:
    return jsonify({"error": "siteNameHierarchy et deviceId sont requis"}), 400
device = find_device(device_id)
if not device:
    return jsonify({"error": f"Device inconnu : {device_id}"}), 404

FABRIC_SITES[site_name] = {
    "siteNameHierarchy": site_name,
    "deviceId": device_id,
    "roles": FABRIC_ROLES.get(device_id, []),
}
return jsonify({"status": "success", "site": FABRIC_SITES[site_name]})

@app.route("/dna/intent/api/v1/business/sda/fabric-site", methods=["GET"])
def list_fabric_sites():
    if not check_auth_token():
        return jsonify({"error": "Invalid or missing X-Auth-Token"}), 401
    return jsonify({"response": list(FABRIC_SITES.values())})

@app.route("/dna/intent/api/v1/business/sda/hostonboarding/user-device", methods=["POST"])
def onboard_host():
    if not check_auth_token():
        return jsonify({"error": "Invalid or missing X-Auth-Token"}), 401
    body = request.get_json(force=True)
    device_id = body.get("deviceId")
    vn_name = body.get("virtualNetworkName")
    interface_name = body.get("interfaceName")

    device = find_device(device_id)
    if not device:
        return jsonify({"error": f"Device inconnu : {device_id}"}), 404
    if vn_name not in VALID_POOLS:
        return jsonify({"error": f"Reseau virtuel inconnu : {vn_name}. Attendu : {list(VALID_POOLS)}"}), 400
    if device["reachabilityStatus"] != "Reachable":
        return jsonify({"error": f"Device {device['hostname']} injoignable, onboarding impossible"}), 409

    entry = {
        "device": device["hostname"],
        "interfaceName": interface_name,
        "virtualNetworkName": vn_name,
        "ipPool": VALID_POOLS[vn_name],
    }
    ONBOARDED_HOSTS.append(entry)
    return jsonify({"status": "success", "onboarding": entry})

```

```

@app.route("/dna/intent/api/v1/business/sda/fabric-site/health", methods=["GET"])
def fabric_health():
    if not check_auth_token():
        return jsonify({"error": "Invalid or missing X-Auth-Token"}), 401

    report = []
    for site_name, site in FABRIC_SITES.items():
        device = find_device(site["deviceId"])
        issues = []
        if not device:
            issues.append("Device de la fabric introuvable dans l'inventaire")
        elif device["reachabilityStatus"] != "Reachable":
            issues.append(f"{device['hostname']} ({'/'}.join(site['roles'])) injoignable")

        report.append({
            "site": site_name,
            "status": "HEALTHY" if not issues else "DEGRADED",
            "issues": issues,
        })
    return jsonify({"response": report})

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=6000, debug=False)
EOF

```

Chaque device se voit attribuer un rôle de fabric : siege-router joue les rôles CONTROL_PLANE_NODE (sait où se trouve chaque hôte) et BORDER_NODE (passerelle vers l'extérieur) — réservé au site central ; agence1/agence2 jouent le rôle EDGE_NODE (point d'entrée des utilisateurs).

Étape 3 — Créer le client SD-Access

```

cat > scripts/netops_client/sda_client.py << 'EOF'
from .dnac_client import DNACClient, DNACError

class SDAClient(DNACClient):
    """Client SD-Access, etend le client DNA Center avec les operations de fabric."""

    def create_fabric_site(self, site_name_hierarchy, device_id):
        url = f"{self.base_url}/dna/intent/api/v1/business/sda/fabric-site"
        return self._post(url, {"siteNameHierarchy": site_name_hierarchy, "deviceId": device_id})

    def list_fabric_sites(self):
        url = f"{self.base_url}/dna/intent/api/v1/business/sda/fabric-site"
        return self._get(url)

```

```

def onboard_host(self, device_id, interface_name, virtual_network_name):
    url = f"{self.base_url}/dna/intent/api/v1/business/sda/hostonboarding/user-device"
    body = {
        "deviceId": device_id,
        "interfaceName": interface_name,
        "virtualNetworkName": virtual_network_name,
    }
    return self._post(url, body)

def fabric_health(self):
    url = f"{self.base_url}/dna/intent/api/v1/business/sda/fabric-site/health"
    return self._get(url)

def _post(self, url, body):
    import requests
    response = requests.post(url, headers=self._headers(), json=body, timeout=self.timeout)
    if response.status_code not in (200,):
        raise DNACError(response.json().get("error", response.text))
    return response.json()

def _get(self, url):
    import requests
    response = requests.get(url, headers=self._headers(), timeout=self.timeout)
    if response.status_code != 200:
        raise DNACError(response.json().get("error", response.text))
    return response.json()
EOF

cat > scripts/netops_client/__init__.py << 'EOF'
from .dnac_client import DNACClient, DNACError
from .sda_client import SDAClient

__all__ = ["DNACClient", "DNACError", "SDAClient"]
EOF

```

Étape 4 — Créer le script de déploiement de la fabric

```

cat > scripts/deploy_sda_fabric.py << 'EOF'
import os
import sys

from netops_client import SDAClient, DNACError

DNAC_URL = os.environ.get("DNAC_URL", "http://localhost:6000")
DNAC_USER = os.environ.get("DNAC_USER", "admin")
DNAC_PASS = os.environ.get("DNAC_PASS")

if not DNAC_PASS:

```

```

print("ERREUR : variable d'environnement DNAC_PASS non definie.")
sys.exit(1)

FABRIC_SITES = [
    ("Global/NexaCorp/Siege", "dev-siege"),
    ("Global/NexaCorp/Agence1", "dev-agence1"),
    ("Global/NexaCorp/Agence2", "dev-agence2"),
]

HOSTS_TO_ONBOARD = [
    ("dev-siege", "GigabitEthernet0/1", "VN_CORP"),
    ("dev-agence1", "GigabitEthernet0/1", "VN_CORP"),
    ("dev-agence2", "GigabitEthernet0/1", "VN_GUEST"),
]

def main():
    client = SDAClient(base_url=DNAC_URL, username=DNAC_USER, password=DNAC_PASS)
    client.authenticate()
    print("Authentifie sur DNA Center.\n")

    print("=== Creation des sites de la fabric SD-Access ===")
    for site_name, device_id in FABRIC_SITES:
        result = client.create_fabric_site(site_name, device_id)
        roles = "/".join(result["site"]["roles"])
        print(f" {site_name} -> {device_id} ({roles})")
    print()

    print("=== Onboarding automatise des hotes ===")
    for device_id, interface, vn in HOSTS_TO_ONBOARD:
        try:
            result = client.onboard_host(device_id, interface, vn)
            info = result["onboarding"]
            print(f" OK : {info['device']}/{info['interfaceName']} -> {info['virtualNetworkName']} ({info['ipPool']})")
        except DNACError as e:
            print(f" ECHEC sur {device_id}/{interface} : {e}")
    print()

    print("=== Verification de la sante de la fabric ===")
    health = client.fabric_health()["response"]
    degraded = False
    for site in health:
        print(f" {site['site']} : {site['status']}")
        for issue in site["issues"]:
            print(f" -> {issue}")
            degraded = True

    if degraded:

```

```

        sys.exit(1)

if __name__ == "__main__":
    try:
        main()
    except DNACError as e:
        print(f"Erreur : {e}")
        sys.exit(1)
EOF

```

Étape 5 — Tester le déploiement complet (tout fonctionne)

```

nohup python scripts/mock_dnac_api.py > /tmp/dnac.log 2>&1 &
sleep 2
export DNAC_PASS=dnacpass123
python scripts/deploy_sda_fabric.py

```

Résultat attendu :

Authentifie sur DNA Center.

```

=== Creation des sites de la fabric SD-Access ===
Global/NexaCorp/Siege -> dev-siege (CONTROL_PLANE_NODE/BORDER_NODE)
Global/NexaCorp/Agence1 -> dev-agence1 (EDGE_NODE)
Global/NexaCorp/Agence2 -> dev-agence2 (EDGE_NODE)

=== Onboarding automatise des hotes ===
OK : siege-router/GigabitEthernet0/1 -> VN_CORP (10.30.0.0/24)
OK : agence1-router/GigabitEthernet0/1 -> VN_CORP (10.30.0.0/24)
OK : agence2-router/GigabitEthernet0/1 -> VN_GUEST (10.30.99.0/24)

=== Verification de la sante de la fabric ===
Global/NexaCorp/Siege : HEALTHY
Global/NexaCorp/Agence1 : HEALTHY
Global/NexaCorp/Agence2 : HEALTHY

```

Si vous avez déjà un mock DNA Center du TP15 qui tourne encore (par exemple sur un ancien terminal oublié), le nouveau nohup échouera à se lier au port 6000 déjà occupé, et c'est l'ancienne version (sans routes SD-Access) qui continuera de répondre — ce qui provoque des erreurs JSON-DecodeError côté client (le serveur renvoie une page d'erreur HTML au lieu de JSON). Libérez le port avant de relancer :

```
netstat -ano | grep ":6000" | grep LISTENING | awk '{print $5}' | sort -u | while read pid; do taskkill //F //PID "$pid"; done
```

Et rappelez-vous : **modifier le fichier mock_dnac_api.py ne suffit pas** si le serveur tourne déjà — Python a chargé le code en mémoire au démarrage, il faut le redémarrer pour qu'il prenne en compte le changement.

Étape 6 — Simuler une panne de nœud (troubleshooting avancé)

```
sed -i 's/"id": "dev-agence1", "hostname": "agence1-router", "managementIpAddress": "10.10.10.2", "reachabilityStatus": "Reachable"/"id":  
"dev-agence1", "hostname": "agence1-router", "managementIpAddress": "10.10.10.2", "reachabilityStatus": "Unreachable"/' scripts/mock_dnac_api.py
```

Redémarrez le mock (voir l’encart ci-dessus) puis relancez :

```
python scripts/deploy_sda_fabric.py
```

Résultat attendu :

```
=== Onboarding automatisé des hotes ===
```

```
OK : siege-router/GigabitEthernet0/1 -> VN_CORP (10.30.0.0/24)
```

```
ECHEC sur dev-agence1/GigabitEthernet0/1 : Device agence1-router injoignable, onboarding impossible
```

```
OK : agence2-router/GigabitEthernet0/1 -> VN_GUEST (10.30.99.0/24)
```

```
=== Verification de la sante de la fabric ===
```

```
Global/NexaCorp/Siege : HEALTHY
```

```
Global/NexaCorp/Agence1 : DEGRADED
```

```
-> agence1-router (EDGE_NODE) injoignable
```

```
Global/NexaCorp/Agence2 : HEALTHY
```

L’onboarding échoue précisément sur le site concerné (les autres continuent), et le rapport de santé identifie le rôle exact du nœud en panne.

Étape 7 — Réparer

```
sed -i 's/"id": "dev-agence1", "hostname": "agence1-router", "managementIpAddress": "10.10.10.2", "reachabilityStatus": "Unreachable"/"id":  
"dev-agence1", "hostname": "agence1-router", "managementIpAddress": "10.10.10.2", "reachabilityStatus": "Reachable"/' scripts/mock_dnac_api.py
```

Redémarrez le mock une dernière fois avant de committer.

Étape 8 — Committer et pousser

```
git add scripts/mock_dnac_api.py scripts/netops_client/sda_client.py scripts/netops_client/__init__.py scripts/deploy_sda_fabric.py  
git commit -m "TP16 : fabric SD-Access simulée (sites, onboarding, sante)"  
git push -u origin <prenom>-<nom>-jour4-tp16
```

Ouvrir une pull request <prenom>-<nom>-jour4-tp16 → dev.

Comprendre le TP

Une fabric SD-Access organise les équipements par rôle, pas juste par emplacement géographique : le CONTROL_PLANE_NODE sait “qui est où” dans toute la fabric, le BORDER_NODE fait la passerelle vers l’extérieur, l’EDGE_NODE est le point où les utilisateurs se connectent réellement. Un même site peut cumuler plusieurs rôles (le siège cumule contrôle et bordure ici), ce qui reflète bien une architecture hub-and-spoke où le site central a plus de responsabilités.

L’onboarding automatisé évite de configurer un VLAN à la main sur chaque switch d’accès : on dit simplement “cet hôte doit rejoindre le réseau virtuel VN_CORP”, et la fabric se charge de traduire ça en configuration réseau réelle — c’est la vraie promesse de SD-Access, l’intention plutôt que le détail.

Le lien entre onboarding et santé de la fabric illustre pourquoi le troubleshooting doit être automatisé : un nœud en panne ne casse pas juste “une connexion”, il empêche l’onboarding de tout nouvel hôte sur son site, et ce lien de cause à effet doit être visible immédiatement dans le rapport de santé.

Validation

- ☐ `python scripts/deploy_sda_fabric.py` crée 3 sites de fabric avec les bons rôles
- ☐ Les 3 hôtes sont onboardés avec succès sur leur réseau virtuel respectif
- ☐ La vérification de santé montre HEALTHY sur les 3 sites en temps normal
- ☐ Un nœud rendu injoignable (après redémarrage du mock) fait échouer précisément l’onboarding de son site et apparaît DEGRADED avec le bon rôle identifié
- ☐ Une pull request <prenom>-<nom>- jour4-tp16 → dev est ouverte